Basic Sorting

DSA27 — Lecture 09 · Data Structures and Algorithms

Dr. Haitham A. El-Ghareeb

Faculty of Computers and Information Sciences, Mansoura University

Week 9 · Fall 2026

Where We Are

Sorting buys searching

Week 8: on **sorted** data, a search reads 20 values out of a million.

k searches on n values	Cost
linear search, k times	$O(kn)$
sort once, then binary search k times	sort $+O(k \log n)$

Today: **where the sorted data comes from — and what it costs.**

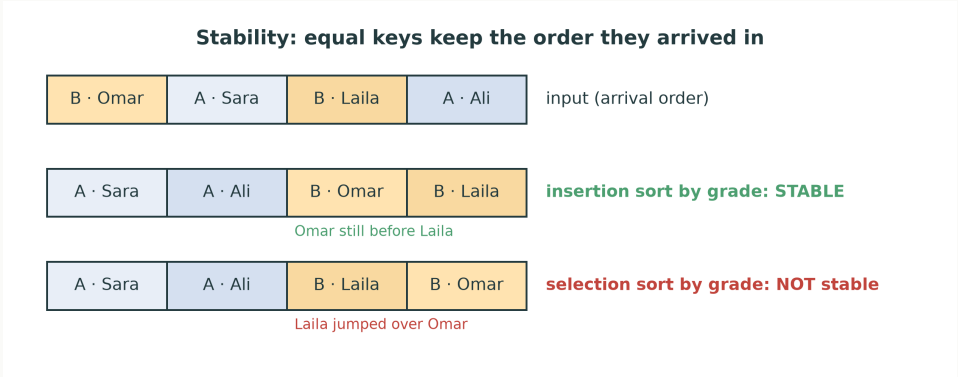
Today

1. Two words first: **in place** and **stable**
2. Bubble, selection and insertion sort — $O(n^2)$ each, but not the same
3. Watching an algorithm: the `_steps` generator — and its price
4. Best, worst, average: comparisons and writes, **measured**
5. Counting sort: sorting without comparing
6. Why no comparison sort can beat $n \log n$

Two Words Before Any Algorithm

In place, and stable

- **In place:** only $O(1)$ extra memory besides the array being sorted.
- **Stable:** elements with **equal keys** keep the order they arrived in.



Why stability matters

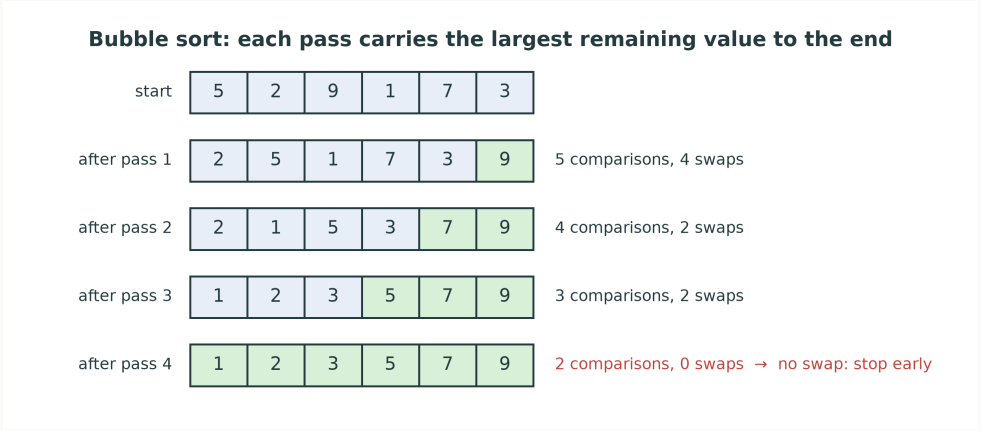
Sort a class list **by name**, then **by grade** with a stable sort:

- the result is by grade, and **by name within each grade**, for free.

This is how spreadsheets sort by two columns, and how every database orders `ORDER BY grade, name`.

Three $O(n^2)$ Sorts

Bubble sort



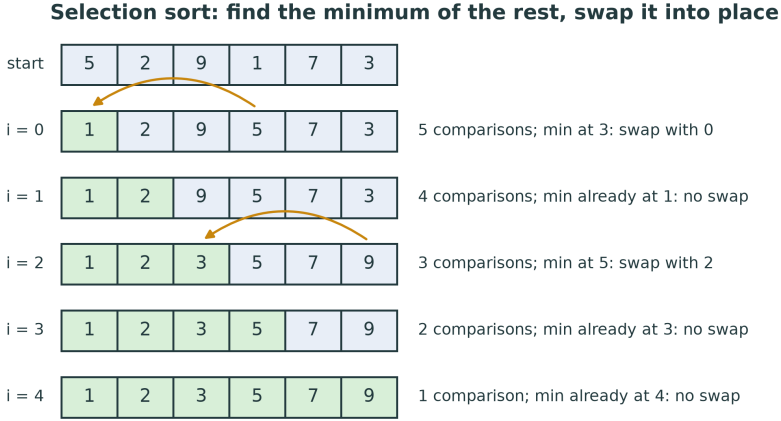
Compare **neighbours**; swap them if they are out of order. Each pass carries the largest remaining value to the end. A pass with **no swap** means sorted: stop.

The code, written as a generator

```
def bubble_sort_steps(values, snapshot=list):
    a = _copy(values)
    n = len(a)
    yield snapshot(a), ()
    for end in range(n - 1, 0, -1):
        swapped = False
        for j in range(end):
            yield snapshot(a), (j, j + 1)
            if a[j] > a[j + 1]:
                a[j], a[j + 1] = a[j + 1], a[j]
                swapped = True
        if not swapped:
            break
    yield snapshot(a), ()

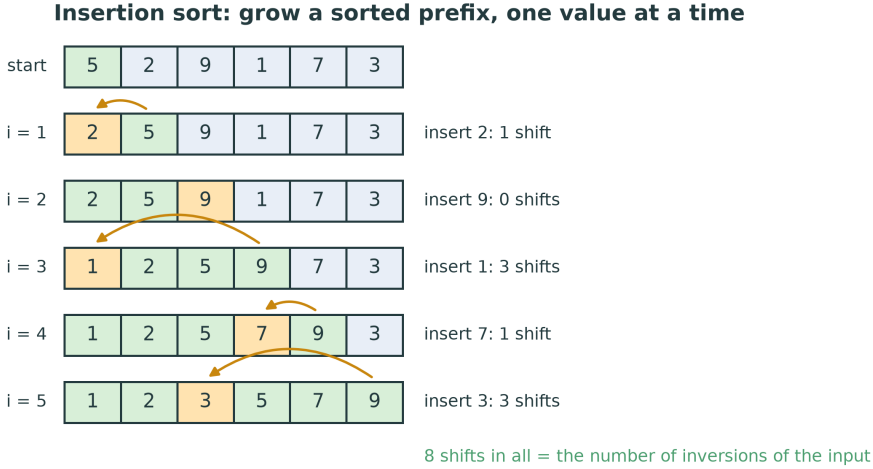

def bubble_sort(values):
    return _finish(bubble_sort_steps(values, snapshot=_live))
```

Selection sort



Find the **minimum** of the unsorted part; **swap** it into the next slot. Always $n(n-1)/2$ comparisons — but at most $n-1$ swaps.

Insertion sort



Grow a **sorted prefix**. Take the next value; shift every larger value in the prefix one slot

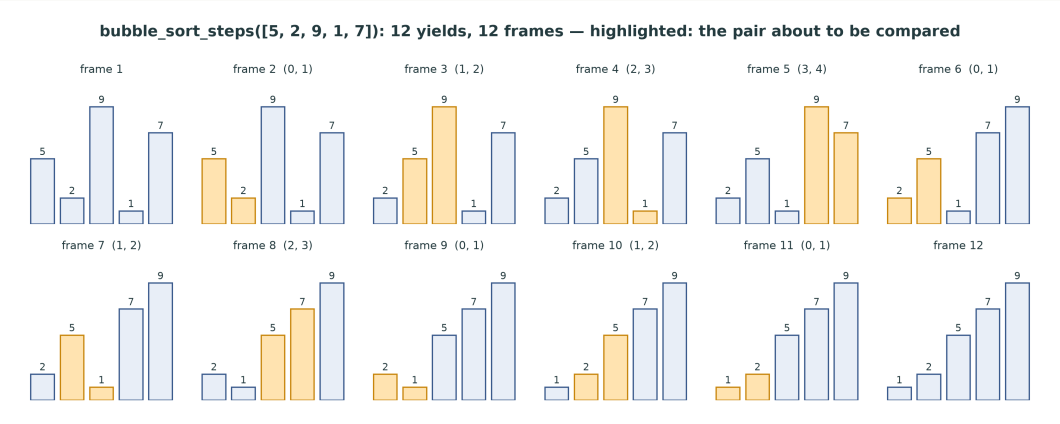
Insertion sort counts inversions

An **inversion** is a pair $i < j$ with $a[i] > a[j]$ — a pair in the wrong order.

- Each shift fixes **exactly one** inversion.
- So insertion sort does **one shift per inversion**: 8 for the example.
- Sorted: 0 inversions. Reversed: $n(n-1)/2$. Random: about $n(n-1)/4$.

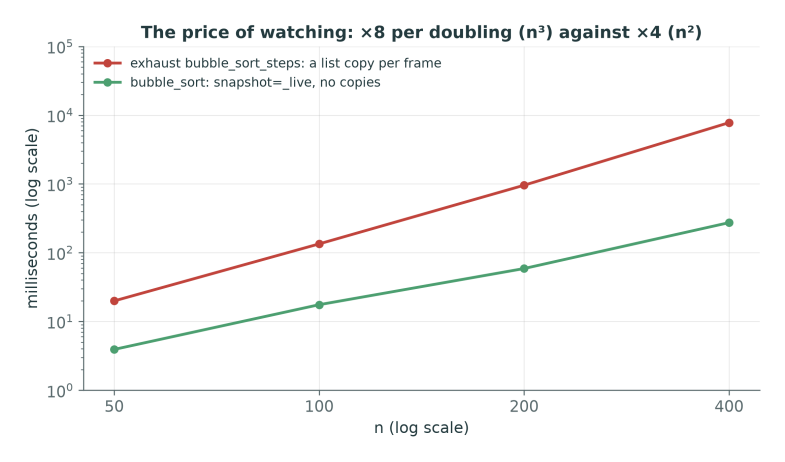
Watching an Algorithm

One yield, one frame




```
from viz.animate import step_slider, animate_bars
step_slider(bubble_sort_steps([5, 2, 9, 1, 7]))     # scrub by hand
animate_bars(bubble_sort_steps([5, 2, 9, 1, 7]))    # play it
```

The price of watching

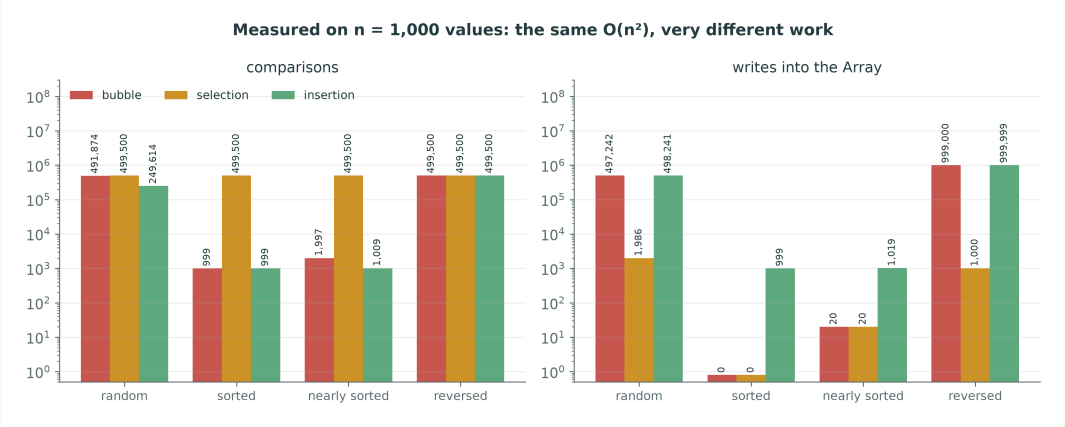


A copy per frame is $O(n)$. Copy before each of $n^2/2$ comparisons: $O(n^3)$.

The plain sort passes `snapshot=_live` — no copies — and is $O(n^2)$ again.

Best, Worst, Average

Counting comparisons and writes

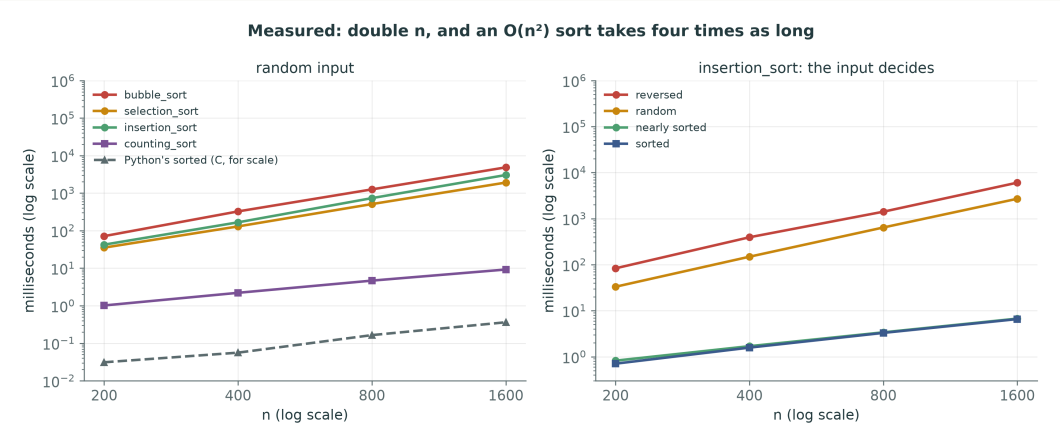


The table

	best	average	worst	writes	stable	in place
bubble (early exit)	n	n^2	n^2	I swaps	yes	yes
selection	n^2	n^2	n^2	$\leq n-1$ swaps	**no**	yes
insertion	n	n^2	n^2	I shifts	yes	yes
counting	$n+k$	$n+k$	$n+k$	$2n$	—	**no**, $O(n+k)$

I = the number of inversions: 0 when sorted, $n(n-1)/2$ when reversed.

Measured time



Insertion sort on nearly sorted input

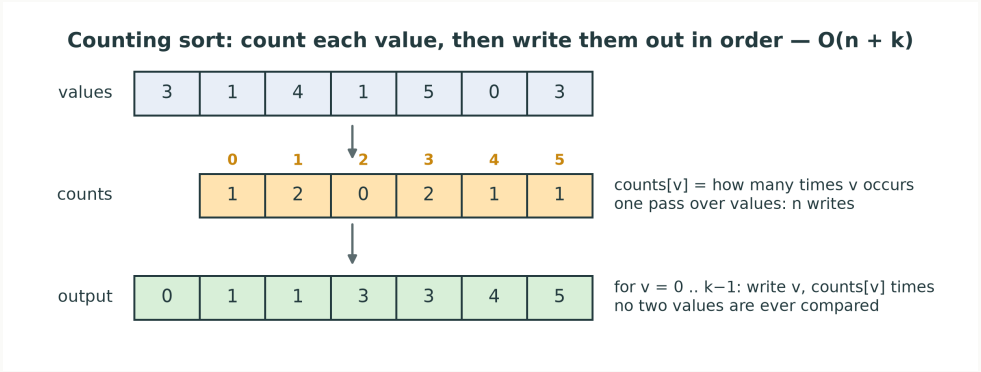
$O(n + I)$: if every value is at most **k** places from home, $I \leq nk$ — **linear** for small k.

Real sorts use it:

- **Timsort** (Python, Java): insertion sort on short runs.
- **Introsort** (C++): quicksort, then insertion sort on small pieces.

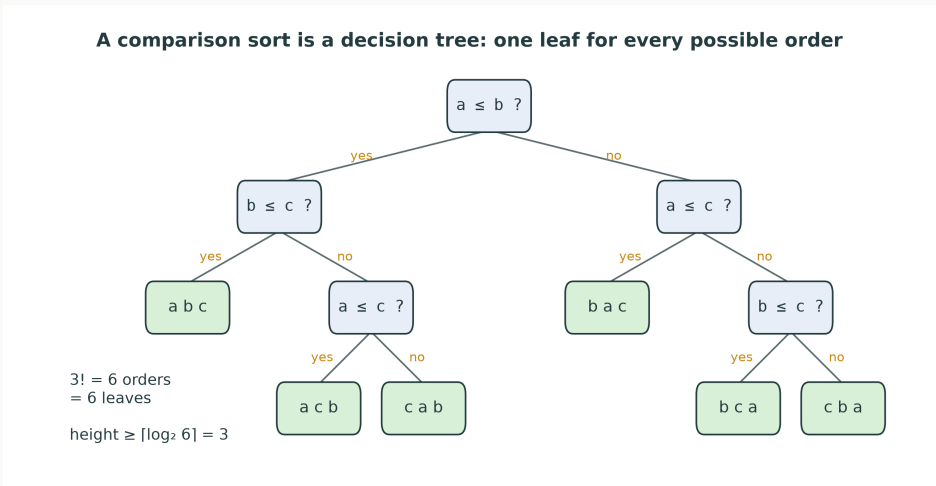
Beyond Comparisons

Counting sort



Values are small non-negative integers, below k: **count** them, then write each value out as many times as it was counted. $O(n + k)$. **No comparisons.**

No comparison sort can beat $n \log n$



n values have **n!** possible orders; the sort must tell them all apart. Each comparison has 2 outcomes. So the worst case needs at least $\log_2 n! \approx n \log_2 n$ comparisons.

How counting sort escapes

The lower bound assumes the sort learns **only by comparing**.

Counting sort never compares: `counts[v] += 1` uses the value **as an address** — one step with k possible outcomes, not 2.

This Week

Exercises: `dsa/sorting.py`

Function	The trap
`bubble_sort_steps`	yield `list(a)`, not `a`; `range(end)`; reset `swapped` each pass
`selection_sort_steps`	compare with `a[smallest]`, not `a[i]`; one swap per round
`insertion_sort_steps`	`j >= 0` **first**; `>` not `>=` (stability)
`bubble_sort`, `selection_sort`, `insertion_sort`	exhaust the generator; copy nothing per step
`counting_sort`	`k = max + 1`; the empty list; the value is the index

```
pytest tests/test_sorting.py -v `
    -k "bubble or selection or insertion or counting or is_sorted"
```

Homework 9 — before Lecture 10

1. **Implement** the four sorts until the 39 tests pass.
2. **Trace** each of the three sorts on `[4, 1, 3, 9, 7, 2]`, one row per pass or round, counting comparisons and swaps or shifts.
3. **Count.** Write a class whose < and > count, sort 1,000 of them sorted, reversed and random, and compare with the figure.
4. **Break it.** Change > to >= in your insertion sort. Which tests fail? Now sort `[(1, 'a'), (1, 'b')]` — why can a list of tuples not show the bug?

Practise online

- **Week 9 quiz**: 22 multiple-choice questions, every answer explained
- **Written practice** on the same page: 5 questions. Write, then mark yourself against the marking points
- In Arabic: week 9

Summary

Seven things to keep

1. Sorting is what makes Week 8's searches possible — **and its cost decides** whether sort-then-search is worth it.
2. **Stable:** equal keys keep their order. Bubble and insertion are; selection is not.
3. Bubble and insertion sort: $O(n)$ **best,** $O(n^2)$ **worst**. Selection sort: $n(n-1)/2$ comparisons always, but at most $n-1$ swaps.
4. Insertion sort does one shift per **inversion**: $O(n+I)$ — near-linear on nearly sorted data, which is why real sorts use it.
5. A `_steps` generator lets you watch your own code — and a copy per frame turns $O(n^2)$ into $O(n^3)$.
6. Counting sort is $O(n+k)$ with **no comparisons** — fast only when k is small.
7. Every comparison sort needs $\log_2 n! = \Theta(n \log n)$ comparisons in the worst case.

Next

Week 10 — Advanced sorting. Merge sort, quicksort and heap sort: three ways to reach the $n \log n$ floor — and why quicksort's pivot decides between $n \log n$ and n^2.